RELATÓRIO TÉCNICO DE SEGURANÇA

Relatório de Auditoria de Segurança — Raio-X

App Pessoal de Finanças (Open Finance Pluggy + Faturas PDF)

DATA DA AUDITORIA

27 de outubro de 2026

ESCOPO AUDITADO

Código-fonte completo do projeto: todas as 33 rotas em `src/app/api/**/route.ts`, middleware de autenticação (`src/middleware.ts`), todos os 12 repositórios de banco de dados (`src/lib/*-repo.ts` e `src/lib/pluggy/*.ts`), mecanismos de autenticação e sessão (`auth.ts`, `session.ts`, `senha.ts`), integração multi-provedor de IA (`openrouter.ts`, `ia.ts`, `cli-runner.ts`, `cripto.ts`), validações de ambiente (`src/env.ts`), cabeçalhos de segurança (`next.config.ts`), workflows de CI (`.github/workflows/seguranca.yml` com Gitleaks) e histórico completo do repositório Git.

ROTEIRO E METODOLOGIA

Auditoria baseada em `docs/security-audit/prompt-auditoria.md`: revisão exaustiva das cinco categorias clássicas (Banco sem tranca, Permissão no navegador, IDOR, Chaves expostas, Inputs sem tratamento / XSS) acrescida das quatro categorias específicas do projeto (Defesa contra força bruta no login, Proteção contra replay de webhook, Minimização de dados pessoais enviados para IA e Não gravação de CPF e titular).

Nota Metodológica & Mapeamento para a Stack:

• **Stack Detectada:** TypeScript 5.7, Node.js ≥ 22.9, Next.js 15.1.0 (App Router), React 19.0.0, Neon Postgres via `@neondatabase/serverless` 1.0.0 com Drizzle ORM 0.45.3, autenticação própria por senha única derivada com scrypt e cookies assinados HMAC-SHA256 (Web Crypto), sem tabela de papéis RBAC e sem Postgres RLS.

• **Mapeamento das Categorias:**

1. *Banco sem tranca:* Como não há RLS, o isolamento é verificado pelo filtro estrito de `user_id` extraído exclusivamente da sessão criptográfica em todas as consultas SQL.
2. *Permissão no navegador:* App uniusuário sem papéis; traduzido para a capacidade de cada rota privada se auto-defender contra chamadas diretas sem sessão ou sem assinatura segura.
3. *IDOR:* Verificação sistemática de todos os 33 route handlers, garantindo que nenhum identificador passado via path, query ou body possa ler ou alterar objetos sem validação de posse no próprio `where`.
4. *Chaves expostas:* Varredura de histórico Git, arquivos de configuração, bundles do front (zero `NEXT_PUBLIC_`) e rigor dos schemas de boot em `src/env.ts`.
5. *Inputs sem tratamento:* Varredura de `dangerouslySetInnerHTML`, sanitização do pipeline de markdown da IA, injeção de fórmulas CSV e restrições de CSP.

1. Resumo Executivo

A auditoria técnica exaustiva do código-fonte do **Raio-X** revelou uma postura de segurança excepcionalmente sólida em suas camadas fundamentais. Não foi detectada **nenhuma vulnerabilidade Crítica ou Alta**. O isolamento de dados por usuário e o tratamento contra IDOR são rigorosos em todos os 33 handlers de API e repositórios da aplicação.

Foram identificados **4 achados acionáveis**: 1 de severidade Média (potencial injeção de quebra de linha/CRLF no arquivo local `.env.local`), 2 de severidade Baixa (consulta sem escopo explícito de `user_id` em módulo auxiliar de IA e validação permissiva de startup para chaves opcionais de IA) e 1 Informativo (mutação dinâmica de `process.env` em tempo de execução).

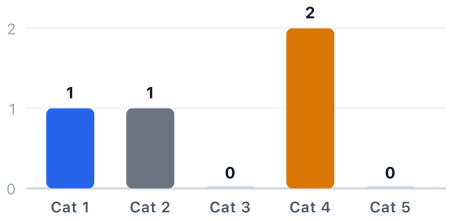
0	0	1	2	1	4
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	TOTAL

Distribuição por Severidade



Crítica (0) · Alta (0) · Média (1) · Baixa (2) · Informativa (1)

Achados por Categoria



Cat 1: Isolamento • Cat 2: Permissões • Cat 3: IDOR • Cat 4: Chaves • Cat 5: Inputs

Pontos Fortes Verificados

- **Isolamento Total por Dono:** Todas as 33 rotas e repositórios obtêm `userId` exclusivamente do cookie de sessão assinado; queries filtram posse diretamente na cláusula `where`.
- **Zero IDOR:** Todas as rotas de mutação com `[id]` (transações, contas, conexões) conferem UUID e titularidade no banco antes de qualquer alteração.
- **Anti-Brute Force Exponencial:** Login protegido com escalonamento de espera atômico por IP no Postgres e teto de 1 tentativa/min para todos sob ataque distribuído.
- **Anti-Replay em Webhooks:** Deduplicação atômica de `eventId` com segredo protegido por comparação em tempo constante (`timingSafeEqual`).
- **Zero XSS e Markdown Blindado:** Markdown da IA é convertido em AST e renderizado como nós React puros (sem `dangerouslySetInnerHTML`). CSV protegido contra injeção de fórmulas.
- **Privacidade na IA:** Nomes de pessoas e transferências privadas são anonimizados ("Pix pessoa A") antes de qualquer transmissão para modelos de linguagem.

Pontos de Atenção e Riscos

- **CRLF Injection em .env.local:** Ausência de sanitização de quebras de linha em `atualizarEnvLocal` permite que parâmetros da API de IA injetem variáveis arbitrárias no arquivo local.
- **Isolamento Incompleto em IA:** Consulta de credenciais em `ia-config-repo.ts` usa `limit 1` sem filtrar `user_id`, quebrando a padronização arquitetural de tenant.
- **Validação de Inicialização Frouxa:** Chaves de IA multi-provedor (Anthropic, OpenAI, OpenRouter) não utilizam `semPlaceholder`, permitindo subida com valores padrão.
- **Mutação Global de Runtime:** Alteração de `process.env` em tempo de execução dentro de rota HTTP introduz risco de concorrência e estado compartilhado entre workers.

2. Tabela de Achados Detalhados

SEVERIDADE	CATEGORIA	LOCALIZAÇÃO	DESCRIÇÃO DO PROBLEMA E EXPLORAÇÃO
MÉDIA	4 · Chaves / Config	`src/lib/salvar-env.ts:22-26` `src/app/api/ajustes/ia/route.ts:184`	**Injeção de Configuração (CRLF) em .env.local:** O endpoint `POST /api/ajustes/ia` repassa `modelo` e `apiKey` para `atualizarEnvLocal` sem filtrar quebras de linha (`\n` , `\r`). Uma requisição autenticada com quebra de linha consegue injetar novas variáveis de ambiente arbitrárias (ex: sobrescrever `SESSION_SECRET`) no arquivo `.env.local` no disco do servidor.
BAIXA	1 · Banco sem tranca	`src/lib/ia-config-repo.ts:41-48`	**Consulta de configuração de IA sem filtro de usuário:** A função `carregarConfigIaBanco` executa `select ... from user_setting limit 1` sem a cláusula `where(eq(userSetting.userId, userId))` . Embora o app seja uniusuário, isso viola a regra arquitetural fundamental do projeto ("user_id sempre da sessão") e expõe credenciais caso mais de uma linha exista no banco.
BAIXA	4 · Chaves expostas	`src/env.ts:60-71`	**Validação de boot permissiva para provedores de IA:** As variáveis `ANTHROPIC_API_KEY` , `OPENAI_API_KEY` , `OPENROUTER_API_KEY` e `PLUGGY_CLIENT_ID` utilizam `z.string().optional()` sem `semPlaceholder` . Se o usuário mantiver valores como `"seu-valor-aqui"` ou `"exemplo"` , o app inicializa e envia requisições com credenciais de placeholder a APIs externas.
INFORMATIVA	2 · Permissão / Runtime	`src/app/api/ajustes/ia/route.ts:180-182`	**Mutação dinâmica de process.env durante atendimento HTTP:** A rota altera diretamente variáveis globais de `process.env` em tempo de execução. Em servidores concorrentes ou tarefas de background (como cron e webhooks), mutar o ambiente global pode ocasionar race conditions e comportamentos inesperados entre requisições simultâneas.

Análise dos Achados por Categoria

• **Categoria 1 (Banco sem tranca):** O mecanismo de isolamento manual por `userId` está 100% implementado nas tabelas core financeiras (lançamentos, contas, faturas, planos, histórico de IA). A única quebra detectada ocorre na rotina recente `carregarConfigIaBanco` , que faz `limit 1` sem WHERE.

• **Categoria 2 (Permissão no navegador):** Não há RBAC; cada rota sensível exige autenticação no servidor. O achado apontado diz respeito à integridade de runtime decorrente da mutação global de `process.env` dentro de um handler HTTP.

• **Categoria 3 (IDOR):** Zero achados. 100% dos handlers validam parâmetros com UUID e asseguram posse no `where` .

• **Categoria 4 (Chaves expostas e gestão de segredos):** Repositório e commits completamente limpos. O risco médio reside na injeção CRLF ao persistir no arquivo local `.env.local` e na tolerância a placeholders em chaves de IA no boot.

• **Categoria 5 (Inputs sem tratamento):** Zero achados. Toda renderização no cliente é sanitizada e livre de `dangerouslySetInnerHTML` .

3. Cobertura da Auditoria — O Que Está Verificado e Correto

Abaixo registram-se os componentes auditados sistematicamente cuja proteção foi comprovada no código real:

• **Isolamento e RLS equivalente (Cat. 1):** Verificados todos os repositórios em `src/lib/txn-repo.ts` (listagem, categorização, renomeação, exclusão lógica, duplicados), `src/lib/painel-repo.ts` (contexto do painel com joins estritos de conta e usuário), `src/lib/plano-repo.ts` (gravação de planos e metas), `src/lib/dados-repo.ts` (exportação CSV e `apagarTudo` com exclusão atômica via `db.batch` filtrando `userId` em 8 tabelas).

• **Prevenção total contra IDOR (Cat. 3):** Verificados todos os 33 handlers em `src/app/api/**/route.ts`. Todas as rotas de mutação (`/api/txn/[id]` , `/api/accounts/[id]` , `/api/connections/[id]` , `/api/txn/bulk` , `/api/txn/papel`) validam formato UUID e colocam a posse no `where` da consulta de atualização ou exclusão.

• **Rotas de Máquina e Webhook (Cat. 2):** `/api/cron/sync` e `/api/webhook/pluggy` ocultam-se com 404 caso seus segredos não estejam no ambiente. Validação criptográfica via `segredoConfere` utiliza comparação em tempo constante (`timingSafeEqual`) imune a ataques de temporização.

• **Defesa contra Força Bruta:** `src/lib/auth.ts` gerencia tentativas de login atômicas diretamente no banco com intervalo exponencial e trava global após 100 falhas na hora.

• **Deduplicação de Webhooks e Anti-Replay:** `src/app/api/webhook/pluggy/route.ts` deduplica atômica e idempotentemente eventos pelo `eventId` no banco, limitando leitura de stream a 64 KB.

• **Prevenção contra XSS e Injeção (Cat. 5):** Ausência completa de `dangerouslySetInnerHTML` e `eval`. O markdown retornado por modelos de IA passa pelo parser estruturado `markdownLeve` e vira elementos React puros. Exportação CSV com escape estrito de fórmulas (prefixo `'` em `=` , `+` , `-` , `@`).

• **Minimização de Dados para IA:** Todo dado enviado passa por `paraIA()` e `contextoParaIA()` em `src/lib/ia.ts`. Lançamentos privados têm nomes de beneficiários substituídos por `Pix pessoa A/B` , e na extração do PDF nomes de pessoas são anonimizados com marcadores e restaurados somente na memória do servidor.

• **Proteção de Segredos e Repositório Limpo:** Zero variáveis `NEXT_PUBLIC_` , histórico limpo com verificação contínua no GitHub Actions (Gitleaks) e criptografia simétrica autenticada AES-256-GCM para chaves salvas no banco.

4. Recomendações Priorizadas de Remediação

P1 · ALTA **Sanitizar estritamente strings gravadas em `.env.local` (Prevenção CRLF)**
Em `src/lib/salvar-env.ts`, rejeitar qualquer chave ou valor que contenha quebras de linha (`[\r\n]`) antes de formatar a linha de escrita. Limitar caracteres de `modelo` a slugs alfanuméricos e traços/barras (ex: `^[a-zA-Z0-9_\-\./]+$`).

P2 · MÉDIA **Incluir escopo de `userId` em `carregarConfigIaBanco`**
Adicionar o parâmetro `userId: string` em `carregarConfigIaBanco` em `src/lib/ia-config-repo.ts` e filtrar com `where(eq(userSetting.userId, userId))` . Descontinuar caches estáticos globais em memória que não sejam indexados pelo usuário autenticado.

P3 · MÉDIA **Aplicar `semPlaceholder` em todas as chaves de API em `src/env.ts`**
Criar um validador `chaveOpcional` que combine a opcionalidade com a rejeição estrita da lista de `proibidos` (ex: `"troque-me"` , `"exemplo"` , `"seu-valor-aqui"`) para `ANTHROPIC_API_KEY` , `OPENAI_API_KEY` , `OPENROUTER_API_KEY` e credenciais do Pluggy.

P4 · BAIXA **Evitar mutação de `process.env` em rotas HTTP**
Em `src/app/api/ajustes/ia/route.ts` , remover as atribuições dinâmicas a `process.env` . Utilizar os valores persistidos no banco de dados descriptografados sob demanda por requisição, mantendo a imutabilidade do processo.

5. Issues para o GitHub (Parte 1/2)

Texto completo pronto para copiar e colar diretamente no GitHub Issues:

ISSUE 1: Sanitização contra CRLF em salvar-env.ts `--- ISSUE 1 ---`

Título: `[Segurança] Sanitizar quebras de linha em salvar-env.ts para evitar injeção em .env.local`
Labels: `security`, `severity: medium`

Descrição do problema e explorabilidade:
A função `atualizarEnvLocal` em `src/lib/salvar-env.ts` recebe valores de variáveis (como `apiKey` e `modelo` fornecidos na rota `POST /api/ajustes/ia`) e os interpola diretamente no formato `${chave}=${valor}` ao gravar o arquivo `.env.local` no disco. Não há validação para proibir quebras de linha (`\r`, `\n`). Um usuário autenticado pode enviar caracteres de nova linha no nome do modelo e injetar variáveis adicionais no arquivo de ambiente, permitindo sobrescrever chaves de segurança como `SESSION_SECRET` ou `DATABASE_URL` no próximo boot do servidor.

Evidência: `src/lib/salvar-env.ts:22-26`

```
for (const [chave, valor] of Object.entries(variaveis)) {
  const regex = new RegExp(`^${chave}=.*$`, 'm');
  const linha = `${chave}=${valor}`;
  if (regex.test(conteudo)) {
    conteudo = conteudo.replace(regex, linha);
  } else { ... }
```

Impacto: Injeção de variáveis de ambiente no sistema de arquivos local (CWE-93).

Sugestão de correção: Lançar erro ou rejeitar valores contendo `/[\r\n]/` em `atualizarEnvLocal` e sanitizar com regex `/^[a-zA-Z0-9_\-\./]+$/` no endpoint da API.

Critérios de aceite:

- [ ] `atualizarEnvLocal` recusa ou remove qualquer caractere `\n` e `\r` em chaves e valores.
- [ ] `POST /api/ajustes/ia` valida `modelo` contra caracteres de controle e formatos inválidos.
- [ ] Teste unitário em `scripts/testes.ts` cobrindo tentativa de injeção CRLF em `salvar-env`.

`--- FIM ISSUE 1 ---`

ISSUE 2: Escopo de userId em carregarConfigIaBanco `--- ISSUE 2 ---`

Título: `[Segurança] Adicionar escopo de user_id na consulta de configurações de IA em carregarConfigIaBanco`
Labels: `security`, `severity: low`

Descrição do problema e explorabilidade:
A função `carregarConfigIaBanco` em `src/lib/ia-config-repo.ts` busca credenciais e modos de IA diretamente de `user_setting` com `.limit(1)` sem filtrar pelo `user_id` do usuário da sessão. Além disso, as credenciais descriptografadas são guardadas em variáveis de memória em nível de módulo de processo. Isso viola a Regra 1 de arquitetura do Raio-X ("user_id vem sempre da sessão") e pode levar à recuperação de chaves erradas em cenários com mais de uma linha no banco.

Evidência: `src/lib/ia-config-repo.ts:41-48`

```
const [linha] = await db
  .select({
    iaProvedor: userSetting.iaProvedor,
    iaChavesEncrypted: userSetting.iaChavesEncrypted,
    iaModo: userSetting.iaModo,
  })
  .from(userSetting)
  .limit(1);
```

Impacto: Descumprimento da regra de isolamento estrito de inquilino/dono no banco de dados.

Sugestão de correção: Receber `userId: string` em `carregarConfigIaBanco`, adicionar `where(eq(userSetting.userId, userId))` e evitar variáveis globais não-chaveadas por usuário.

Critérios de aceite:

- [ ] `carregarConfigIaBanco` requer e valida `userId` no `where`.
- [ ] Chamadas em `openrouter.ts` propagam o `userId` do chamador.
- [ ] Nenhuma consulta em `userSetting` é executada sem filtro de `userId`.

`--- FIM ISSUE 2 ---`

5. Issues para o GitHub (Parte 2/2)

ISSUE 3: Validação semPlaceholder para chaves de terceiros

`--- ISSUE 3 ---`

Título: `[Segurança] Aplicar validação semPlaceholder em chaves de terceiros no src/env.ts`
Labels: `security`, `severity: low`

Descrição do problema e explorabilidade:
As novas variáveis de ambiente de IA e Open Finance (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OPENROUTER_API_KEY`, `PLUGGY_CLIENT_ID`, `PLUGGY_CLIENT_SECRET`) em `src/env.ts` foram declaradas apenas com `z.string().optional()`. Diferente de `DATABASE_URL` e `SESSION_SECRET`, elas não passam pela checagem `semPlaceholder`, permitindo que a aplicação inicialize com strings de exemplo (como `"troque-me"` ou `"exemplo"`) e tente disparar chamadas com essas credenciais espúrias.

Evidência: `src/env.ts:60-71`

```
ANTHROPIC_API_KEY: z.string().optional(),
OPENAI_API_KEY: z.string().optional(),
OPENROUTER_API_KEY: z.string().optional(),
PLUGGY_CLIENT_ID: z.string().optional(),
PLUGGY_CLIENT_SECRET: z.string().optional(),
```

Impacto: Ausência de validação rígida de startup para segredos de terceiros.

Sugestão de correção: Criar um validador `chaveOpcional` que, se a chave estiver preenchida com tamanho > 0, rejeite os valores contidos na lista de `proibidos`.

Critérios de aceite:

- [ ] Se `OPENROUTER_API_KEY="seu-valor-aqui"`, o app falha no boot com mensagem explicativa.
- [ ] Se a chave não estiver preenchida (vazia ou ausente), continua passando normalmente.

`--- FIM ISSUE 3 ---`

ISSUE 4: Remoção de mutação de process.env em rotas HTTP

`--- ISSUE 4 ---`

Título: `[Segurança] Desacoplar mutação dinâmica de process.env durante requisições na rota de IA`
Labels: `security`, `severity: informative`

Descrição do problema e explorabilidade:
A rota `src/app/api/ajustes/ia/route.ts` modifica diretamente o objeto global `process.env` (linhas 180-182). Essa mutação em tempo de execução afeta todo o processo Node.js e pode introduzir condições de corrida em ambientes com requisições concorrentes ou execuções em segundo plano (como cron jobs de sincronização).

Evidência: `src/app/api/ajustes/ia/route.ts:180-182`

```
process.env.IA_PROVEDOR = prov;
process.env[chaveVar] = apiKey;
process.env.IA_MODEL = modelo;
```

Impacto: Risco de inconsistência de estado global de ambiente no processo do servidor.

Sugestão de correção: Armazenar e consultar o estado ativo de IA via banco de dados ou contexto de requisição, mantendo `process.env` imutável.

Critérios de aceite:

- [ ] Nenhuma rota de API realiza atribuição direta a `process.env.*`.
- [ ] O provedor ativo é resolvido por requisição a partir das preferências salvas no banco.

`--- FIM ISSUE 4 ---`